RISE: Randomized Input Sampling for Explanation of Black-Box Models

Vitali Petsiuk, Abir Das, Kate Saenko — Boston University

BMVC 2018 — arXiv:1806.07421

Presented by **Aritra Dasgupta**

xAI Reading Group

April 24, 2026

Roadmap

1. Motivation & Problem Setting
2. Related Work
3. RISE Algorithm
4. Mathematical Foundations (§3.1)
5. Evaluation Metrics
6. Experimental Results
7. Extensions & Limitations

Why Explainable AI?

The Core Problem

Deep neural networks achieve remarkable performance — but are effectively **black boxes**.

High-stakes domains

Medical diagnosis Autonomous driving
Criminal justice Financial decisions

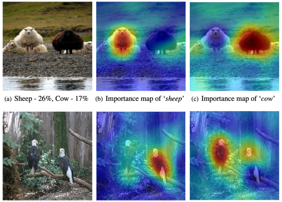
(d) Bird - 100%, Person - 39% (e) Importance map of ‘*bird*’ (f) Importance map of ‘*person*’

ResNet50 confuses parts of a bird for a person. *Why?*

Open questions about any deep neural network:

- *How* did it arrive at this decision?
- *Which* pixels drove the prediction?
- *When* can it be trusted?
- *How* do we fix systematic errors?

Motivating Examples: The Sheep-Cow and Bird-Person Confusion



(a) Sheep - 26%, Cow - 17% (b) Importance map of '*sheep*' (c) Importance map of '*cow*'

(d) Bird - 100%, Person - 39% (e) Importance map of '*bird*' (f) Importance map of '*person*'

Figure 1: Our proposed RISE approach can explain why a black-box model (here, ResNet50) makes classification decisions by generating a pixel importance map for each decision (redder is more important). For the top image, it reveals that the model only recognizes the white sheep and confuses the black one with a cow; for the bottom image it confuses parts of birds with a person. (Images taken from the PASCAL VOC dataset.)

Top prediction

Sheep: 26% Cow: 17%

RISE reveals that the model only activates on **white sheep** for the "sheep" class, and confuses the **black sheep** with a cow — likely due to training data scarcity.

Bird–Person confusion

Bird: 100% Person: 39%

The **left bird's posture** drives the "person" class prediction — a non-intuitive, human-invisible cue.

The Saliency / Importance Map Approach

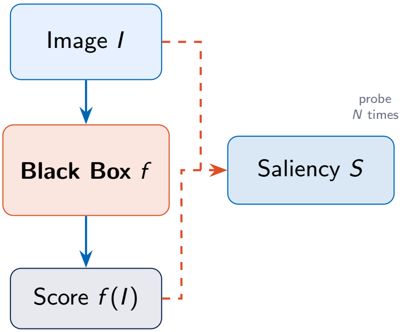
Goal: Given a model f and input image I, produce a **saliency map** $S \in \mathbb{R}^{H \times W}$ where:

$$S(\lambda) \propto \text{“importance of pixel } \lambda \text{ for prediction”}$$

Two broad families

- **White-box**: Uses internal model info — gradients, activations, weights (GradCAM, CAM, ...)
- **Black-box**: Only input/output access. More general, model-agnostic.

RISE is a black-box approach — it treats the network as a pure function $f : \mathcal{I} \to \mathbb{R}$.



Landscape of Explanation Methods

White-Box Methods (require internals)

Method	Requirement
CAM [Zhou et al.]	GAP architecture only
Grad-CAM [Selvaraju et al.]	Gradients + features
Deconv [Zeiler et al.]	Deconvolutional net
MWP/c-MWP [Zhang et al.]	Backprop strategy
Backprop [Simonyan et al.]	Input gradients
Fong & Vedaldi	Perturbation + backprop

Limitation: Architecture-specific, require re-implementation

per network

Black-Box Methods

Method	Approach
LIME [Ribeiro et al.]	Linear approx. on superpixels
Sliding Window [Zeiler]	Occlusion scanning
RISE (ours)	Random masking + MC

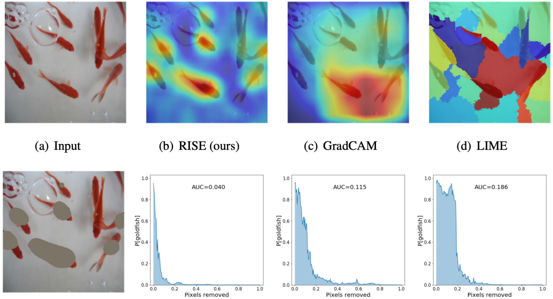
Advantage: Works on any model as-is

LIME's weakness

Superpixel-based saliency does not capture correct importance regions at pixel granularity.

RISE position: Black-box $\cap$ pixel-level $\cap$ any architecture

Visual Comparison: RISE vs. GradCAM vs. LIME



(a) Input (b) RISE (ours) (c) GradCAM (d) LIME

(e) Image during deletion (f) RISE-Deletion (g) GradCAM-Deletion (h) LIME-Deletion

Figure 2: Estimation of importance of each pixel by RISE and other state-of-the-art methods for a base model's prediction along with 'deletion' scores (AUC). The top row shows an input image (from ImageNet) and saliency maps produced by RISE, Grad-CAM [23] and LIME [21] with ResNet50 as the base network (redder values indicate higher importance). The bottom row illustrates the deletion metric: salient pixels are gradually masked from the image (2(e)) in order of decreasing importance, and the probability of the 'goldfish' class predicted by the network is plotted *vs.* the fraction of removed pixels. In this example, RISE provides more accurate saliency and achieves the lowest AUC.

Core Intuition: Masking as Probing

Key Idea

If a pixel is important, hiding it should hurt the model's confidence. If it is irrelevant, masking it should change little.

The RISE approach:

1. Generate N random binary masks $\{M_i\}$
2. Multiply each mask element-wise with the image: $I \odot M_i$
3. Query the black box: obtain scores $f(I \odot M_i)$
4. Aggregate: importance $\propto$ weighted sum of masks, with weights $=$ output scores

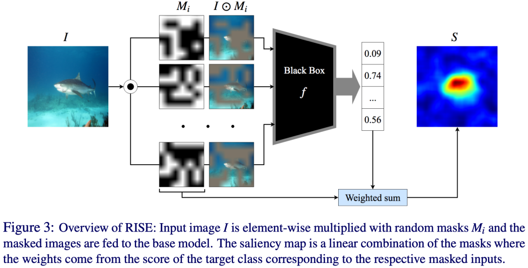
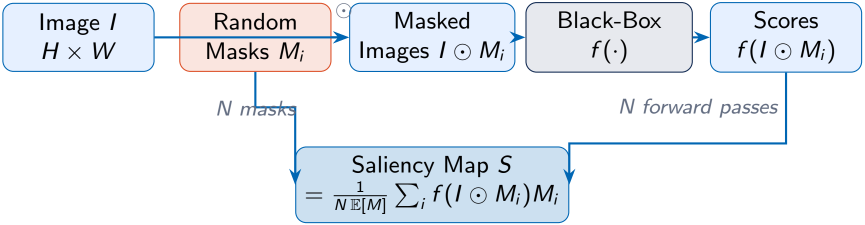


Figure 3 from the paper: RISE overview diagram

No gradients. No internals. No architecture assumptions.

RISE Pipeline Overview



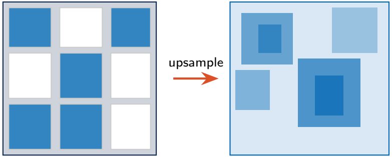
Monte Carlo Approximation

$$\hat{S}_{I,f}(\lambda) = \frac{1}{\mathbb{E}[M] \cdot N} \sum_{i=1}^{N} f(I \odot M_i) \cdot M_i(\lambda)$$

Mask Generation: Why Not Independent Pixels?

Problem with pixel-independent masks:

- Mask space is $2^{H \times W}$ — exponential!
- Sharp binary pixel patterns cause *adversarial artefacts*
- A single pixel flip can radically change a DNN's output



$h \times w$ binary mask Smooth upsampled mask

Bilinear upsampling produces **smooth spatial regions** (not isolated pixels), reducing adversarial artefacts and improving MC convergence.

RISE Solution: Upsample Small Masks

1. Sample binary $h \times w$ mask $(h, w \ll H, W)$ with $P[\text{cell} = 1] = p$
2. Upsample to $(h+1)C_H \times (w+1)C_W$ via **bilinear interpolation**
3. Apply **random shift** by $(0, C_H) \times (0, C_W)$
4. Crop to $H \times W$

Mathematical Setup

Notation:

$$\Lambda = \{1, \ldots, H\} \times \{1, \ldots, W\} \quad \text{(pixel coords)}$$
$$\mathcal{I} = \{I \mid I : \Lambda \to \mathbb{R}^3\} \quad \text{(image space)}$$
$$f : \mathcal{I} \to \mathbb{R} \quad \text{(black-box model)}$$
$$M : \Lambda \to \{0, 1\} \quad \text{(random binary mask, dist. } D\text{)}$$
$$I \odot M \quad \text{(element-wise multiplication)}$$

Definition: Pixel Importance

The **importance** of pixel λ is the *expected model output over all masks, conditioned on pixel λ being* **visible**:

$$S_{I,f}(\lambda) = \mathbb{E}_M\big[f(I \odot M) \mid M(\lambda) = 1\big]$$

Intuition

When pixel λ is unmasked ($M(\lambda) = 1$), how confident is the model on average?

High importance $\Rightarrow$ model is consistently more confident when this pixel is visible.

Low importance $\Rightarrow$ visible or not, the score barely changes.

This is a **causal** notion: we observe the marginal effect of revealing each pixel, averaged over all random contexts.

Derivation: From Conditional Expectation to Weighted Sum

Starting from the definition:

$$S_{I,f}(\lambda) = \mathbb{E}_M\big[f(I \odot M) \mid M(\lambda) = 1\big] = \sum_m f(I \odot m)\, P[M = m \mid M(\lambda) = 1]$$

Step 1 — Expand conditional probability:

$$P[M = m \mid M(\lambda) = 1] = \frac{P[M = m,\ M(\lambda) = 1]}{P[M(\lambda) = 1]}$$

Step 2 — Simplify the joint:

$$P[M = m,\ M(\lambda) = 1] = \begin{cases} 0 & \text{if } m(\lambda) = 0 \\ P[M = m] & \text{if } m(\lambda) = 1 \end{cases} = m(\lambda) \cdot P[M = m]$$

Step 3 — Substitute:

$$S_{I,f}(\lambda) = \frac{1}{P[M(\lambda) = 1]} \sum_m f(I \odot m) \cdot m(\lambda) \cdot P[M = m]$$

Derivation (continued): Matrix Form & Estimation

Step 4 — Recognise the expectation: Since $P[M(\lambda) = 1] = \mathbb{E}[M(\lambda)]$, and the sum is just $\mathbb{E}[f(I \odot M) \cdot M]$

Closed-Form Saliency Map (Equation 5)

$$\boxed{S_{I,f} = \frac{1}{\mathbb{E}[M]} \sum_{m} f(I \odot m) \cdot m \cdot P[M = m] = \frac{\mathbb{E}[f(I \odot M) \cdot M]}{\mathbb{E}[M]}}$$

Step 5 — Monte Carlo Estimator (Equation 6):

$$S_{I,f}(\lambda) \overset{\text{MC}}{\approx} \underbrace{\frac{1}{\mathbb{E}[M] \cdot N}}_{\text{normalisation}} \underbrace{\sum_{i=1}^{N} f(I \odot M_i)}_{\text{confidence weight}} \underbrace{\cdot\, M_i(\lambda)}_{\text{pixel visibility}}$$

Properties

- **Consistent**: converges to true $S_{I,f}$ as $N \to \infty$
- **Normalised**: $\mathbb{E}[M]$ corrects for mask density

Computational cost

Requires N forward passes (4000–8000 for VGG16/ResNet50). This is the primary computational bottleneck.

Deep Dive: Why the Normalisation by $\mathbb{E}[M]$?

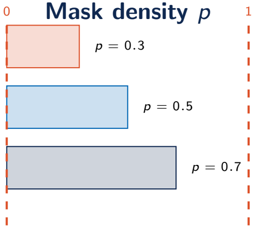
The Problem Without Normalisation

If we use $p = 0.5$ (50% of pixels masked), pixel λ appears in *half* the masks on average.

A pixel that always contributes when visible will be *underweighted* compared to its true importance, because it is only observed in 50% of samples.

Normalisation corrects for:

- The **probability** that any pixel appears in a given mask ($P[M(\lambda) = 1]$)
- For i.i.d. Bernoulli(p) masks: $\mathbb{E}[M(\lambda)] = p$ for all λ
- So we divide by $p \cdot N$



Dividing by $\mathbb{E}[M]$ ensures the importance estimate is **independent of mask density** p.

LIME [Ribeiro et al., 2016]

Also a black-box method. Fits a local linear model g near the input by sampling perturbations and minimising:

$$\min_{g \in G} \sum_{z \in Z} \pi_x(z) \left(f(z) - g(z)\right)^2 + \Omega(g)$$

Importance = coefficients of g. **Superpixel-level**, not pixel-level.

RISE vs. LIME

	RISE	**LIME**
Granularity	pixel	superpixel
Model	non-parametric	linear
Masks	smooth random	segment-based
Bias	none	linear assumption

Shapley Connection

RISE's importance definition is related to **Shapley values** from cooperative game theory — the conditional expectation over all coalitions of pixels that include λ mirrors the Shapley marginal contribution framework.

Why Not Human-Centric Metrics Alone?

Standard approach: Pointing Game

Hit if the *highest-saliency pixel* falls inside the human-annotated bounding box.

$$\text{Accuracy} = \frac{\#\text{Hits}}{\#\text{Hits} + \#\text{Misses}}$$

Fundamental limitation

A model may correctly detect a cow from **grass context** — human annotation marks the cow, not the grass. A good explanation for the model's actual reasoning would be penalised!

RISE Proposal: Causal Metrics

Evaluate explanations by **causal intervention**:

- Remove/add pixels according to the explanation
- Measure model confidence change
- No human annotation required

Two metrics: Deletion & Insertion

Deletion Metric

Algorithm

1. Order pixels by *decreasing* importance (from S)
2. Progressively set them to **constant grey** (not blur)
3. Record f(modified image) at each step
4. Score = **Area Under the Curve (AUC)** — *lower is better*

Algorithm 1

procedure DELETION
    Input: black box f, image I, importance map S, number of pixels N removed per step
    Output: deletion score d
    $n \leftarrow 0$
    $h_n \leftarrow f(I)$
    while I has non-zero pixels **do**
        According to S, set next N pixels in I to 0
        $n \leftarrow n+1$
        $h_n \leftarrow f(I)$
    $d \leftarrow \mathtt{AreaUnderCurve}(h_i \text{ vs. } i/n, \quad \forall i = 0, \ldots n)$
    return d

Deletion algorithm pseudocode (Appendix A)

Intuition: If we first remove the truly important pixels, the model's confidence should *drop sharply and quickly*.

Why constant value (not blur) for deletion?

A good classifier fills in missing detail from context. Blur still leaves low-frequency signal, reducing the drop. Constant ensures the pixel is truly gone.

Insertion Metric

Algorithm

1. Start with a **highly blurred** version of the image
2. Order pixels by *decreasing* importance (from S)
3. Progressively reveal them from the original image
4. Record f(current image) at each step
5. Score = AUC — *higher is better*

Intuition: If we first reveal the truly important pixels, the model's confidence should *rise quickly*.

Why blur (not black canvas) for insertion?

Revealing pixels onto blank canvas creates sharp edges — the oval-shaped region may trigger "balloon". Blurred canvas removes artefacts.

Algorithm 2

procedure INSERTION
  Input: black box f, image I, importance map S, number of pixels N removed per step
  Output: insertion score d
  $n \leftarrow 0$
  $I' \leftarrow \texttt{Blur}(I)$
  $h_n \leftarrow f(I)$
  while $I \neq I'$ **do**
    According to S, set next N pixels in I' to corresponding pixels in I
    $n \leftarrow n+1$
    $h_n \leftarrow f(I')$
  $d \leftarrow \texttt{AreaUnderCurve}(h_i \text{ vs. } i/n, \quad \forall i = 0, \ldots n)$
  return d

Insertion algorithm pseudocode (Appendix A)

Experimental Setup

Datasets

- **PASCAL VOC07**: object classification, pointing game
- **MSCOCO 2014**: multi-label, pointing game
- **ImageNet**: deletion/insertion evaluation

Base Models

- ResNet50 [He et al., 2016]
- VGG16 [Simonyan & Zisserman, 2015]
- Pre-trained, from PyTorch model zoo

RISE Hyperparameters

Parameter	Value
Mask size ($h \times w$)	7×7
Image size ($H \times W$)	224×224
Mask probability (p)	0.5
N (VGG16)	4000
N (ResNet50)	8000

Baselines

- Grad-CAM (white-box)
- LIME (black-box)
- Sliding Window (black-box)
- AM, Deconv, CAM, MWP, c-MWP (white-box)

Results: Deletion and Insertion on ImageNet

Table 1: Comparative evaluation in terms of deletion (lower is better) and insertion (higher is better) scores on ImageNet dataset. Except for Grad-CAM, the rest are black-box explanation models.

Method	ResNet50		VGG16	
	Deletion	Insertion	Deletion	Insertion
Grad-CAM [23]	0.1232	0.6766	0.1087	0.6149
Sliding window [31]	0.1421	0.6618	0.1158	0.5917
LIME [21]	0.1217	0.6940	0.1014	0.6167
RISE (ours)	**0.1076 ± 0.0005**	**0.7267 ± 0.0006**	**0.0980 ± 0.0025**	**0.6663 ± 0.0014**

- RISE achieves **best deletion and insertion** for both ResNet50 and VGG16
- Outperforms even the **white-box** Grad-CAM
- Low standard deviation across 3 runs $\Rightarrow$ robust to mask randomness

Key takeaway

RISE (0.1076 deletion, 0.7267 insertion) vs. Grad-CAM (0.1232, 0.6766) on ResNet50. A black-box method beating a white-box method!

Results: Pointing Game on VOC07 and MSCOCO

Table 2: Mean accuracy (%) in the pointing game. Except for RISE, the rest require white-box model.

Base model	Dataset	AM [25]	Deconv [31]	CAM [33]	MWP [32]	c-MWP [32]	RISE
VGG16	VOC	76.00	75.50	-	76.90	80.00	**87.33** $\pm$ 0.49
	MSCOCO	37.10	38.60	-	39.50	49.60	**50.71** $\pm$ 0.10
Resnet50	VOC	65.80	73.00	**90.60**	80.90	89.20	88.94 $\pm$ 0.61
	MSCOCO	30.40	38.2	**58.4**	46.8	57.4	55.58 $\pm$ 0.51

- **VGG16 on VOC**: RISE achieves **87.33%** — significantly better than all white-box methods (best white-box: c-MWP 80.00%)
- **VGG16 on MSCOCO**: RISE **50.71%** vs. c-MWP 49.60%
- **ResNet50 on VOC**: RISE 88.94%, close to CAM 90.60% (which requires GAP architecture)
- Consistent low variance ($< 0.6\%$) across all settings

Reminder: RISE is the only black-box method in Table 2

All others require white-box access. RISE is competitive despite being strictly more general.

RISE Importance Maps with Deletion/Insertion Curves

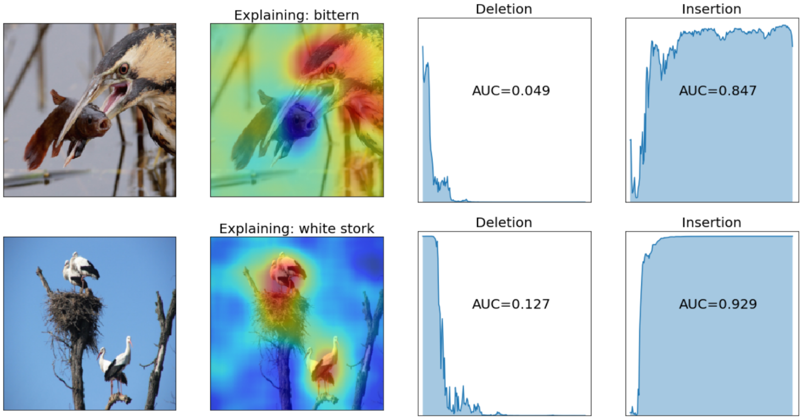


Figure 4: RISE-generated importance maps (second column) for two representative images (first column) with deletion (third column) and insertion (fourth column) curves.

Figure 4 from the paper. Second column: importance maps (redder = more important). Third/Fourth columns:

Extension: RISE for Image Captioning

Setup: Captioning model
$f(I, s, w_k) = P[w_k \mid I, w_1, \ldots, w_{k-1}]$

RISE Extension

For each word w_k in the caption, compute:

$$S_{w_k}(\lambda) = \frac{1}{N\,\mathbb{E}[M]} \sum_{i=1}^{N} f(I \odot M_i, s, w_k) \cdot M_i(\lambda)$$

This produces a **per-word saliency map**.

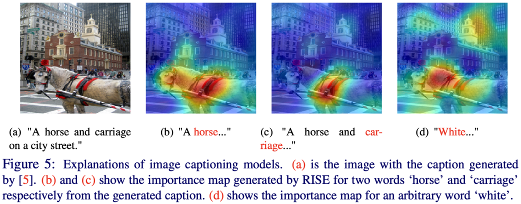


Figure 5: For the caption "A horse and carriage on a city street", RISE correctly highlights the horse region for "horse" and the carriage region for "carriage".

Advantage over prior work:
Not constrained to fixed-size patches (unlike Ramanishka et al.). Better handles varying object sizes.

Failure Cases and Limitations

Known Failure Modes

1. **Background noise**: Monte Carlo approximation with finite N leaves residual noise, especially around object boundaries
2. **Small objects**: Hard to isolate with 7×7 masks; importance can leak to surroundings
3. **High computational cost**: $N = 4000$–8000 forward passes per image
4. **Multi-object scenes**: Importance maps can be diffuse when multiple relevant objects overlap

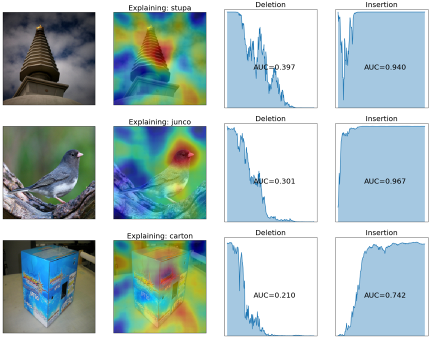


Figure 9: Failure cases. In some cases RISE does pick up more important features, but cannot get rid of the background noise (in part due to MC approximation with only a subset) like in rows 1 and 2.

Appendix Figure 9: Failure cases — rows 1 & 2 show background noise due to MC approximation. The model picks up features but cannot cleanly isolate them.

Summary

RISE in One Sentence

Probe the model with randomly masked images, then reconstruct pixel importance as a weighted average of the masks, weighted by the model's output on each masked image.

The key contributions:

1. **RISE algorithm**: Black-box pixel saliency via Monte Carlo masking
2. **Smooth mask generation**: Bilinear upsampling + random shift avoids adversarial artefacts
3. **Causal evaluation metrics**: Deletion and Insertion — human-agnostic, objective
4. **State-of-the-art**: Outperforms black-box methods; competitive with white-box
5. **Captioning extension**: Natural generalisation to sequence models

Key References

- Petsiuk, Das, Saenko. **RISE: Randomized Input Sampling for Explanation of Black-box Models**. BMVC 2018. `arXiv:1806.07421`
- Selvaraju et al. **Grad-CAM**. ICCV 2017.
- Ribeiro, Singh, Guestrin. **LIME: "Why Should I Trust You?"**. KDD 2016.
- Zhou et al. **CAM: Learning Deep Features for Discriminative Localization**. CVPR 2016.
- Zeiler & Fergus. **Visualizing and Understanding CNNs**. ECCV 2014.
- Zhang et al. **Top-down Neural Attention by Excitation Backprop**. IJCV 2017.
- Fong & Vedaldi. **Interpretable Explanations of Black Boxes by Meaningful Perturbation**. ICCV 2017.
- He et al. **Deep Residual Learning**. CVPR 2016.
- Simonyan & Zisserman. **VGGNet**. ICLR 2015.

Thank you!

Aritra Dasgupta
xAI Reading Group